

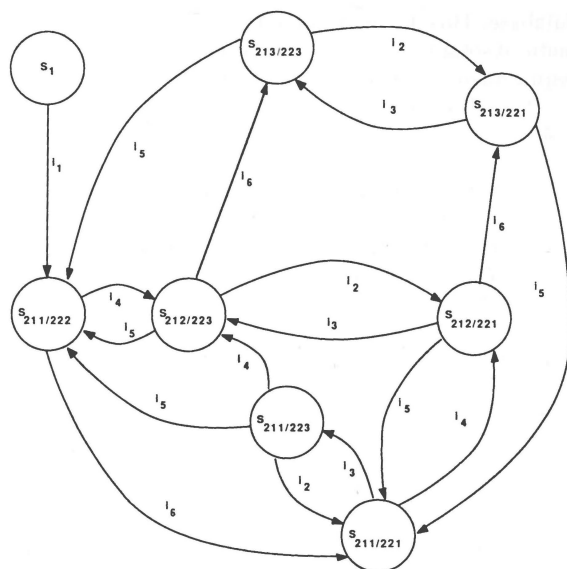


FIGURE 15. An Equivalent Conventional STD

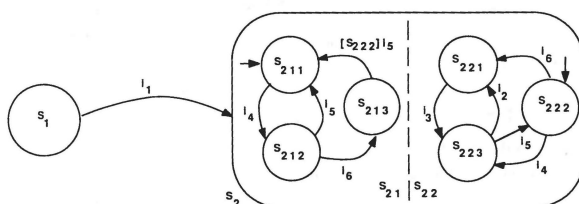


FIGURE 16. Specifying Transitions Dependent on States

an R-net as a column of the state transition matrix, and thus the R-net is simply an organizational piece of a full FSM. R-net notation is shown in Figure 17. Figure 18 shows an example of an R-net for the "Dial 9" column of Figure 4. Figure 19 defines the same behavior in RSL.

REVS has been operational since 1977. Originally hosted on a Texas Instruments Advanced Scientific Computer and on a CDC 7600, and later rewritten in PASCAL and rehosted on VAX, REVS has most recently been rewritten in Ada. TRW in Huntsville and Ascent Logic Corporation in San Jose have extended SREM and REVS to encompass earlier development phases, such as a system level requirements and system design. The new support tool is called the Distributed Computing Design System (DCDS), and the methodology is called the System Level Requirements Methodology (SYSREM) [3].

### Requirements Language Processor

The Requirements Language Processor (RLP) [20, 26] is part of an overall Requirements Processing System (RPS) developed by GTE Laboratories in Waltham, Massachusetts. The motivation for developing RLP was similar to that of REVS: namely that monolithic FSMs seemed to be too unwieldy for the requirements specification of complex, real-time systems. Whereas REVS

uses the R-net as the organizational unit of the SRS, RLP uses the *stimulus-response sequence*. In essence, a stimulus-response sequence is a trace of a two-way dialog between the system under specification and its environment. The selection of which sequences to use is simple: use sequences that define *typical* dialogs. These typical dialogs should correspond to actual user-oriented, user-known, external system features. Figure 20 shows a complete example of the external behavior of a local call [23].

The decision of whether or not to use RLP on a particular application should be based on the types of questions a typical customer or user would ask about

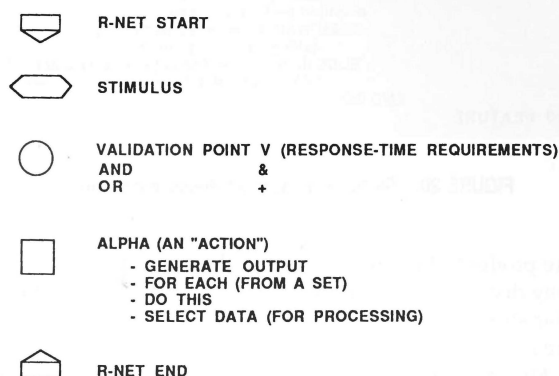


FIGURE 17. R-Net Notation

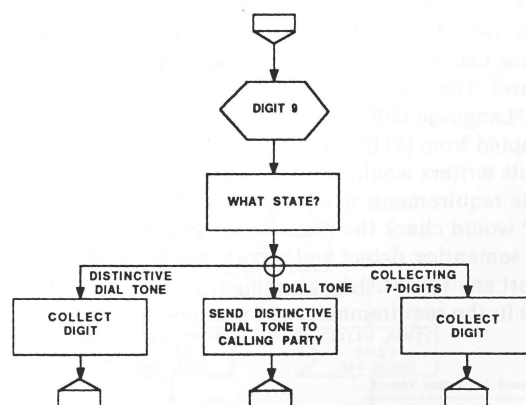


FIGURE 18. An R-Net Example

```

R Net: Process_digit_nine
Structure:
  Input_interface digit_9_dialed_by_calling_party
  What_state
  Do (state = dial tone)
    Send_distinctive_dial_tone_to_calling_party
  (state = collecting 7 digits)
    Collect_digit
    Terminate
  (distinctive dial tone)
    Collect_digit
    Terminate
  End
End

```

FIGURE 19. An RSL Example

```

FEATURE: local to local call
  IN idle state
  WAIT FOR calling party to go off hook
  IN receiving dial tone state
  WAIT FOR calling party to go on hook OR dial local no.
  IF calling party is on hook
  THEN IN idle state
  ELSE IF called party is busy
  THEN IN receiving busy tone state
    WAIT FOR calling party to go on hook
    IN idle state
  ELSE IN receiving ringback tone state
    WAIT FOR calling party to go on hook OR
    called party to go off hook
    IF calling party is on hook
    THEN IN idle state
    ELSE
      DO WHILE (called party is off hook)
        Connect (called party, calling party)
        WAIT FOR called party to go on hook OR
        calling party to go on hook
        IF called party is on hook
        THEN WAIT FOR called party to go off hook OR
        calling party to go on hook
        ELSE IN disconnected calling party state
          WAIT FOR called party to go on hook
      END DO
    END DO
END FEATURE

```

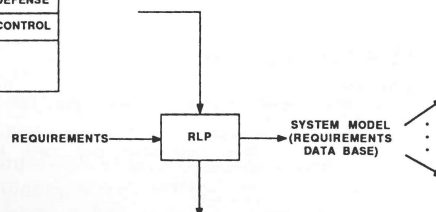
FIGURE 20. Stimulus-Response Sequence Example

the product. If the questions are about features (e.g., long-distance calls, call-forwarding) rather than particular stimuli (e.g., dialing 0, dialing 9), then RLP may be the answer.

Also, RLP is *multilingual* [23, 52], which means that it supports a wide variety of requirements languages, each tailored to a specific application. For example, it might be desirable to use RLP for four different applications: battlefield command and control, patient monitoring, ballistic missile defense, and nuclear reactor control. The four languages used would be defined in the "Language definition tables" as shown in Figure 21 (adapted from [21]). Once these tables existed, requirements writers would select an appropriate language, write requirements using that language, and run RLP. RLP would check the requirements for correct syntax and semantics; detect and report violations; detect and report any inconsistency, ambiguity, and incompleteness in the requirements; and generate a requirements

## LANGUAGE DEFINITION TABLES

|                           |
|---------------------------|
| BATTLEFIELD C2            |
| PATIENT MONITORING        |
| BALLISTIC MISSILE DEFENSE |
| NUCLEAR REACTOR CONTROL   |
|                           |
|                           |



## REPORTS

- ERRORS (INCONSISTENT, INCOMPLETE, AMBIGUOUS)
- FORMATTED OUTPUTS

FIGURE 21. The Requirements Language Processor Architecture

database. This database can later be used to drive automatic system test generation [15, 19, 21], automated requirements simulation [22], and automatic program synthesis [24]. RLP also has the capability of generating SDL charts from textual requirements languages.

## The Specification and Description Language

The *Specification and Description Language* (SDL) [46] was developed in the late 1970s by CCITT for the external behavior and internal design of telephone switching systems. SDL is a superset of STDs. Figure 22 shows the SDL for the identical segment of an SRS as was shown in Figure 20 using RLP.

## PAISLey

The *Process-oriented, Applicative and Interpretable Specification Language* (PAISLey) was developed by Pamela Zave while at the University of Maryland and later at AT&T Bell Laboratories. PAISLey [60, 62] is a language for the requirements specification of embedded systems using an operational approach. *Operational* means that the resulting specification can be executed (or interpreted), and the resulting behavior would mimic the behavior required of the system to be built. From that perspective, statecharts, REVS, and RLP all provide operational approaches to requirements specification.

What makes PAISLey unique is that it is a simple language, with rigor and formality adopted from the disciplines of asynchronous processes and functional programming. *Asynchronous processes*, exemplified by the cooperating "and" FSMs of Harel, are independently operating autonomous abstractions of computations that usually communicate with each other via some protocol, such as interrupts, messages, handshakes, etc. *Functional programming* is a technique that describes behavior through the definition of functions that map inputs into outputs rather than through the definition of procedures that, when executed, transform inputs into outputs.

When using PAISLey, the requirements writer decomposes both the system under specification and its environment into sets of asynchronous interacting processes. For example, Figure 23 shows a decomposition of a patient monitoring system into five processes and its environment into four processes (adapted from [60]). Once this is accomplished, each process must be defined. This is accomplished by defining the range of possible states in which the process may enter (i.e., the state space) and by declaring and defining sets of functions that define how the processes change state (successor functions) and how the processes interact (exchange functions). Processes that are part of the environment are treated no differently than processes that are part of the system. Figure 24 shows a small subset of some of these definitions and declarations of processes and functions. Figure 24 is only a partial specification of the behavior of the patient monitoring system shown in Figure 23. Still necessary are descriptions of the update of the acceptable vital sign tolerances for

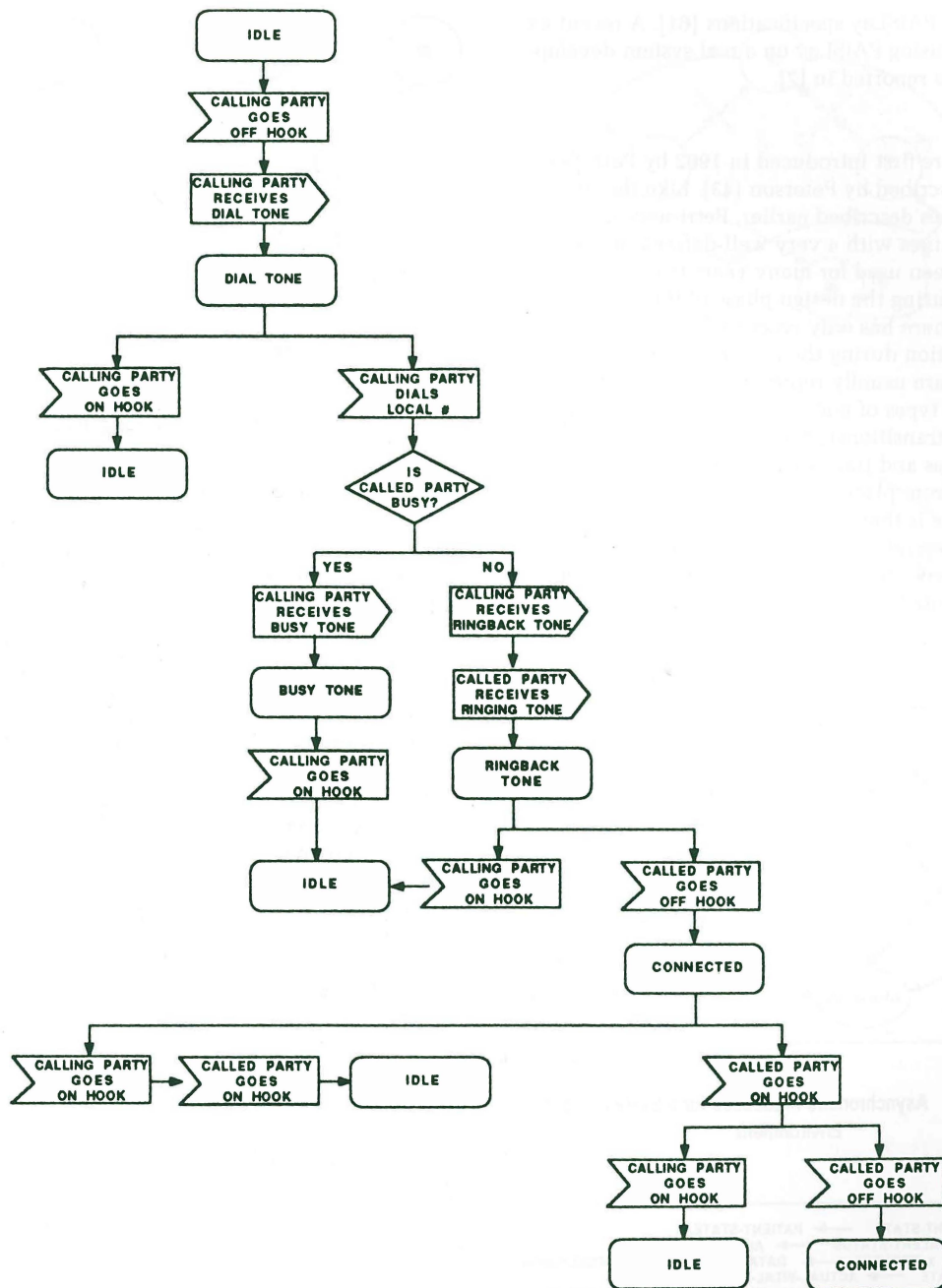


FIGURE 22. An SDL Example

patients, the action of the real-time clock as an initiator of the "patient-check-cycle" and as a source of the current time for the database storage, and many other functions.

Some critics of PAISLey claim that the resulting specification is a design specification, not a requirements specification. Two primary reasons for this claim are that the resulting specification is difficult, if not impossible, for the typical customer or user to understand, and that writing in PAISLey requires a lot of rigorous,

detailed thought. Zave defends PAISLey against this claim [60] by saying that customers and users need not view the same representation of the requirements and that the difference between requirements and design is subtle. Further she claims that the software synthesis process is a series of iterations in which each level implies the requirements for the next level.

A set of PAISLey tools has recently been implemented to find and report syntactic or semantic violations, perform static analysis, and provide interactive

execution of PAISley specifications [61]. A recent experiment in using PAISley on a real system development effort is reported in [7].

### Petri-nets

*Petri-nets* were first introduced in 1962 by Petri [44] and later described by Peterson [43]. Like the finite state machines described earlier, Petri-nets are abstract virtual machines with a very well-defined behavior. They have been used for many years to specify process synchrony during the design phase of time-critical applications. There has only recently been interest in their application during the requirements phase.

Petri-nets are usually represented as a graph composed of two types of nodes: circles (called places) and lines (called transitions). Arrows (directed arcs) interconnect places and transitions. Black dots (called tokens) move from place to place according to a specific rule. The rule is that tokens may pass through a transition only when (a) a clock pulse has arrived (i.e., the Petri-net is synchronized) and (b) all the arrows entering that transition are emanating from places that contain tokens.

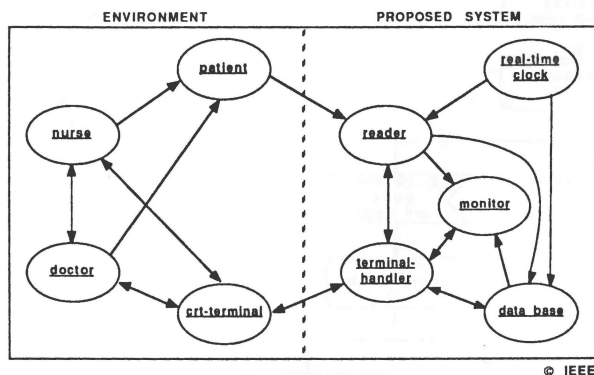


FIGURE 23. Asynchronous Processes for a System and Its Environment

```
(patient-cycle: PATIENT-STATE → PATIENT-STATE,
patient-check-cycle: ALERT-STATUS → ALERT-STATUS
retrieve: DATABASE x REQUEST → DATABASE x VITAL-SIGN-TOLERANCE
sense: PATIENT-STATE → ACTUAL-VITAL-SIGNS
check: VITAL-SIGN-TOLERANCE x ACTUAL-VITAL-SIGNS → ALERT-STATUS
update: DATABASE x ACTUAL-VITAL-SIGNS → DATABASE)
```

(a) Partial Declaration of Processes and their Successor Functions

```
patient-check-cycle[n] =
  proj-2-1[(check[(sense[n], retrieve[patient-database,n]),
    update[(patient-database, sense[n])]])]
```

(b) Definition of the Primary Patient Monitoring System Function

FIGURE 24. Sample PAISley Statements

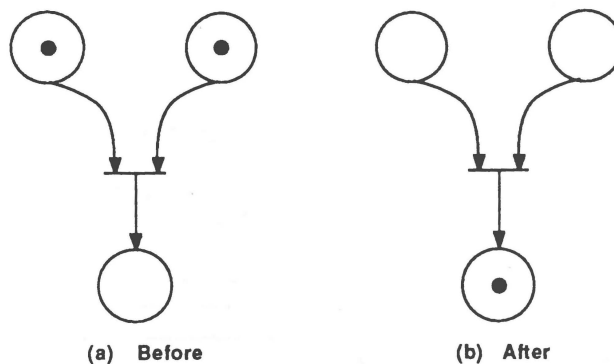


FIGURE 25. Token Merging in Petri Nets

Let us assume that only one token is permitted at any one place.<sup>1</sup> The arrival of a clock pulse at Figure 25(a) will result in Figure 25(b). Also, let us assume that, in the cases of a fan-out, the token will be replicated and a copy will follow every path that it can.

Let us now observe the behavior of a Petri-net through a series of clock pulses over a period of time. The initial state of the Petri-net is shown in Figure 26(a). Figures 26(b) through (e) show the movements of the tokens as each clock pulse is processed.

Let us now look at a specific application in which a Petri-net could be helpful during the requirements phase: the automation of a warehouse. Figure 27(a) shows a Petri-net that captures the timing requirements of this application. At place  $P_1$ , a truck arrives at the loading dock. At place  $P_2$ , the paperwork is processed and inventory is checked. At place  $P_3$ , all the conveyor belts are moving from the loading dock to the warehouse. The belts are transporting people, robots, and forklifts. At place  $P_4$ , the conveyor belts are moving from the warehouse to the loading dock with all the requisite merchandise. At place  $P_5$ , the goods are loaded onto the truck.

This is a fine use of a Petri-net and at first glance explicitly shows how more than one truck can be handled at a time. On closer examination, however, there is a problem: if one truck is being processed at  $P_4$  and the next truck is being processed at  $P_3$ , the conveyor belts would need to be going in both directions at once—an obvious impossibility. The solution is to introduce three new places, as shown in Figure 27(b). In particular, place  $P_7$  prevents  $P_3$  and  $P_4$  from being active simultaneously. I call the token in  $P_7$  the *escort token*, because it escorts tokens representing trucks (i.e., those on the  $P_1$  to  $P_5$  track) along the critical region of  $P_3$  and  $P_4$ . As soon as the conveyor belts have completed the return of all the merchandise to the loading dock, not only does the token at  $P_4$  move to the next step,  $P_5$ , but  $P_7$  becomes reactivated and ready to escort the next token through the critical region.  $P_6$  and  $P_8$

<sup>1</sup> This is not the case with pure Petri-nets, but is an extension put forth by Patil [59].

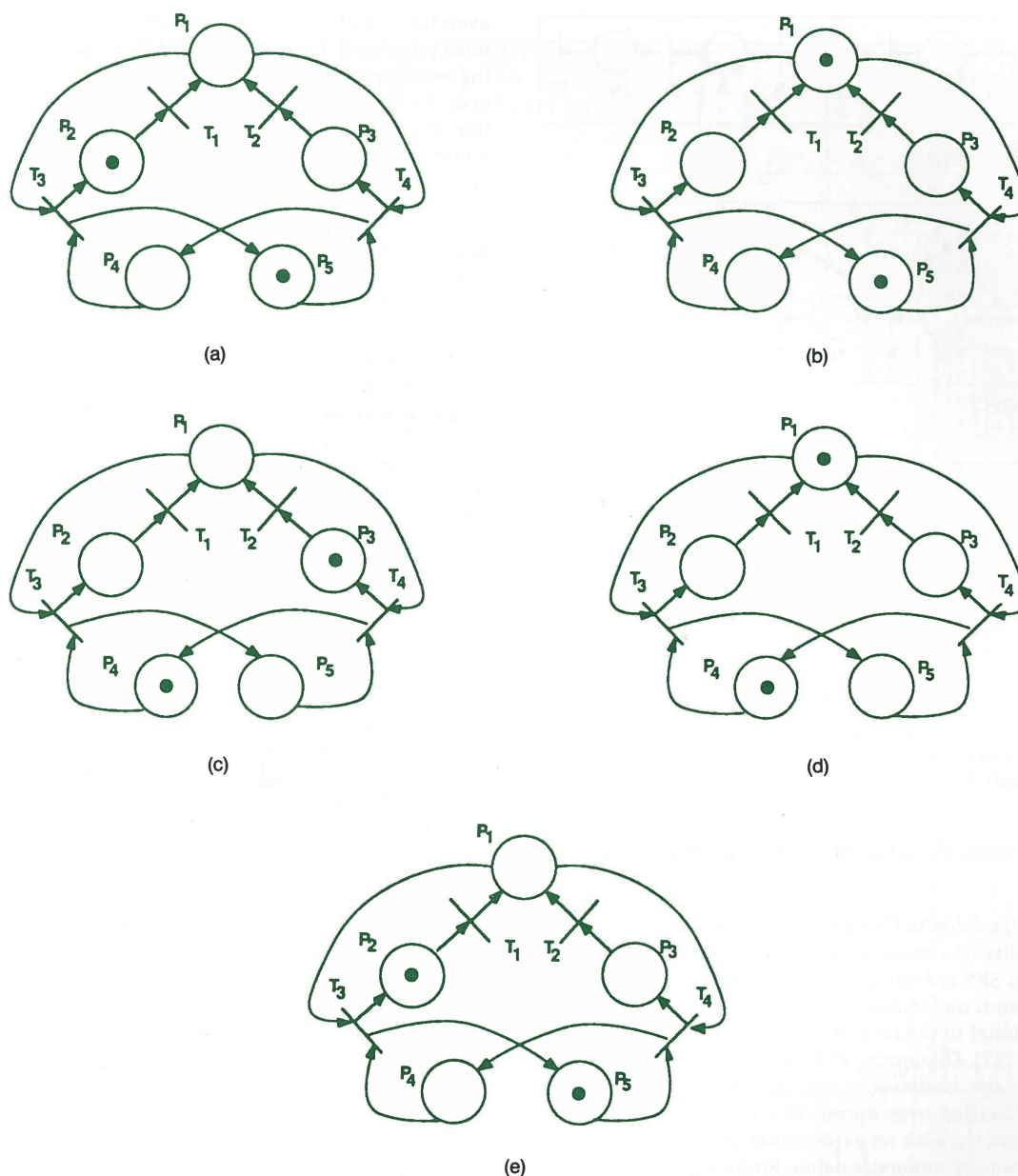


FIGURE 26. A Petri Net Sequence Example

serve similar functions for single steps; they prevent two tokens from accidentally falling into the same station.

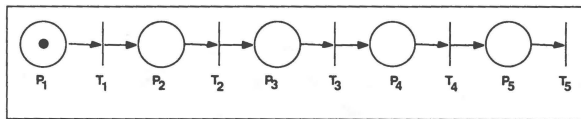
Presumably, this entire Petri-net would also need places corresponding to physical buttons (see Figure 27(c)): the  $P_9$  button would be pressed by the order taker when paperwork was complete, the  $P_{10}$  button would be pressed by the loaders when the conveyor belt was fully loaded with merchandise, and the  $P_{11}$  button would be pressed by the unloaders when the conveyor belt was completely unloaded.

Once again, Petri-nets are relatively simple to understand and are best used to describe pieces of intended system behavior where ambiguity cannot be tolerated

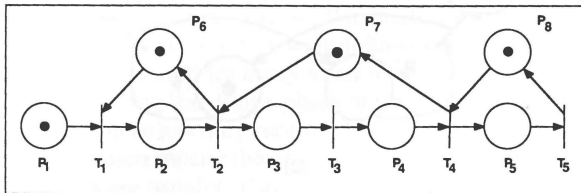
and precise process synchrony is important. One of the best descriptions of how Petri-nets can be used for real applications is given in [1]. Extensive examples of applying Petri-nets for the requirements specification of real-time systems are found in [59].

#### COMPARISON OF TECHNIQUES

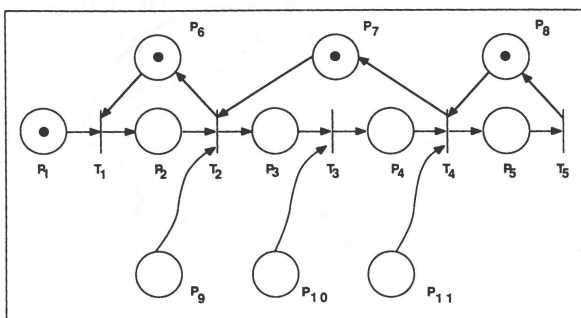
In an earlier section, eight criteria were defined for the evaluation of requirements specification techniques. The techniques will now be analyzed, compared, and contrasted using each of the eight criteria. Figure 28 summarizes the results and scores each approach and criteria on a scale from 0 (poor) to 10 (excellent).



(a) Version 1



(b) Version 2



(c) Version 3

FIGURE 27. A Petri Net Example for a Warehouse

### Understandable to Computer-naïve Personnel

The ability of a computer-naïve customer to understand an SRS is a subjective determination. On the other hand, understandability appears to be inversely proportional to the level of complexity and formality present [23]. Obviously, PDL, which is primarily natural language, is almost as easy to read as natural language. Decision trees appear in the non-technical literature regularly with no explanation, so they too are extremely understandable. Finite state machines, SA/RT, REVS, RLP, and SDL all require about the same amount of instruction to computer-naïve people to enable them to read and comprehend SRS sections written using those approaches.

I have successfully taught hundreds of non-computer-oriented personnel to comprehend at least the basics of all these approaches in less than an hour per approach. Statecharts, PAISLey, and Petri-nets appear to be much more difficult to comprehend than the others for three unrelated reasons. In particular, statecharts are probably difficult because of their semantic richness. The concepts that statecharts add to basic FSMs provide expressive power and reduce the size of the resulting specification; many of the new concepts, however, are not intuitive to the non-computer scientist (albeit 100 percent intuitive to any computer scientist with

even basic skills in automata theory and asynchronous processes). Conceptually, PAISLey suffers from the same problem that statecharts do. Practically, however, PAISLey has one additional problem: language that is difficult to understand for the noncomputer scientist. Petri-nets are difficult to understand by an applications-oriented non-computer expert.

### Basis for Design and Test

For a technique to make an SRS more useful to designers and system testers, its ambiguity level must be lowered and its understandability to the computer-oriented people who design and test must be increased. The former issue has already been discussed. For the latter issue, computer-oriented people prefer formal, precise documents. I contend that the very reasons why some techniques received low points for ambiguity are the same for why they received low points for understandability from testers and designers.

### Automated Checking

Miller and Taylor [40] define three distinct levels of errors detectable in an SRS:

- (1) *Static Structural*. Signals transmitted through inappropriate ports, or signals received by an entity but not sent by any entity;
- (2) *Behavioral*. Two requirements conflict; a specification is ambiguous, incomplete, or redundant (see Davis [27] for further discussion of this class of error); and
- (3) *Protocol*. Deadlocks between subsystems (see Sunshine [51] for further discussion of this class of error).

Natural language does not provide for automatic checking of any of these. PDL will be capable of detecting static structural errors. Finite state machines (using an automated tool, such as the Transition Diagram Editor [55-57]), SA/RT, REVS, and RLP all provide automatic checking of behavioral and static structural errors. Statecharts (using an automated tool such as STATEMATE<sup>2</sup> [34, 50]) and PAISLey both provide full protocol error checking in addition to behavioral and static structural checking. Pure Petri-nets show synchrony very well, but one needs the Yoeli and Barzelai extensions [59] to fully specify protocols between asynchronous entities. Unfortunately, to my knowledge Petri-nets have not been automated, and thus suffer somewhat in their ability to automatically check for protocol violations. Decision trees and decision tables do not provide the ability to specify protocol and, although they are used to specify behavior, I do not know of any tools that automatically check for consistency. Finally, SDL can be used to specify protocol. To my knowledge, however, SDL has not been automated to any degree.

<sup>2</sup> STATEMATE1 is a trademark of I-Logix, Inc.

| Criterion                                        | Natural Language                                                                                    | FSM's                                                                                                                                   | DT's                                                                   | PDL                                                                                     | SA/RT                                                                                                                                                           | Statecharts                                                                                                             | REVS                                                                                                | RLP                                                                                           | SDL                                                                                              | PAISLEY                                                                                                                         | Petri-nets                                                                                                       |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| 1<br>Understandable to computer-native personnel | 10<br>Obviously understandable to computer-native personnel                                         | 10<br>Requires an hour or two of time to understand by computer-native personnel                                                        | 10<br>Appear in literature regularly without explanation               | 10<br>Almost as easy to read as natural language because most of it is natural language | 7<br>See FSM's                                                                                                                                                  | 5<br>Many FSM extensions not intuitive to the noncomputer scientist                                                     | 7<br>See FSM's                                                                                      | 7<br>See FSM's                                                                                | 7<br>See FSM's                                                                                   | 2<br>Many concepts are not intuitive to the noncomputer scientist; hard-to-understand language for same audience                | 4<br>Synchrony and Petri-nets appear to be difficult for noncomputer scientists to grasp                         |
| 2<br>Basic Error Design and Test                 | 0<br>With no formality there is no basis                                                            | 7<br>The formal model enables one to unambiguously define intended product behavior and thus serves as a basis for both design and test | 7<br>See FSM's                                                         | 2<br>Being little more than natural language, the lack of formality provides no basis   | 7<br>The static data-oriented view of a system is helpful for data-intensive applications. See also FSM's                                                       | 9<br>With the additional power over FSM's, there is ambiguity and thus a more formal basis for design and test          | 9<br>See statecharts                                                                                | 9<br>See statecharts                                                                          | 7<br>See FSM's                                                                                   | 9<br>See statecharts                                                                                                            | 7<br>See FSM's                                                                                                   |
| 3<br>Automated Checking                          | 0<br>No processor is available to perform any checking                                              | 7<br>Static structural and behavioral errors can be detected                                                                            | 0<br>See natural language                                              | 1<br>Static structural errors can be detected                                           | 7<br>See FSM's                                                                                                                                                  | 9<br>Static structural, behavioral, and protocol errors can be detected                                                 | 7<br>See FSM's                                                                                      | 7<br>See FSM's                                                                                | 0<br>See natural language (potential exists for static structural and behavioral error checking) | 9<br>See statecharts                                                                                                            | 0<br>See natural language (potential exists for protocol error checking)                                         |
| 4<br>External View, Not Internal View            | 0<br>No help provided                                                                               | 3<br>Needs human discipline in defining external entities up front                                                                      | 3<br>See FSM's                                                         | 0<br>No help provided                                                                   | 2<br>For many applications, the design of the software                                                                                                          | 5<br>Structure view is not explicitly and independently of behavioral views                                             | 7<br>Same as RLP, but simpler because R-Net is more design oriented than stimulus-response sequence | 8<br>Once external entities are defined, specifier stays naturally in the requirements domain | 8<br>See RLP                                                                                     | 3<br>A common criticism of PAISLEY is that design orientation. Zave is not concerned if design is performed during requirements | 2<br>Very difficult to use Petri-nets in the applications domain. Many people versed in Petri-nets are designers |
| 5<br>SRS Organizational Assistance               | 1<br>Paragraph and Chapter                                                                          | 2<br>The machine                                                                                                                        | 2<br>The table or tree                                                 | 2<br>The module                                                                         | 8<br>Hierarchical definition of transformations and machines                                                                                                    | 9<br>See SA/RT; plus more semantic information; hierarchical alternatives; multiple viewpoints                          | 8<br>The R-Net plus hierarchies or alphabets                                                        | 7<br>The stimulus-response sequence; the feature                                              | 7<br>The stimulus-response sequence; the feature                                                 | 9<br>See statecharts                                                                                                            | 2<br>The net                                                                                                     |
| 6<br>Prototype Generation                        | 0<br>None                                                                                           | 10<br>Yes, using IDE tools                                                                                                              | 0<br>None                                                              | 0<br>None                                                                               | 0<br>None                                                                                                                                                       | 10<br>Yes, using logic tools                                                                                            | 7<br>Yes, using TRW auxiliary coding                                                                | 2<br>Feasibility demonstrated by not implemented                                              | 0<br>None                                                                                        | 10<br>Yes, using Bell Labs tools                                                                                                | 0<br>None                                                                                                        |
| 7<br>Automatic Test Generation                   | 0<br>None                                                                                           | 0<br>None                                                                                                                               | 0<br>None                                                              | 0<br>None                                                                               | 0<br>None                                                                                                                                                       | 0<br>None                                                                                                               | 0<br>None                                                                                           | 10<br>Yes, using GTE tools                                                                    | 0<br>None                                                                                        | 0<br>None                                                                                                                       | 0<br>None                                                                                                        |
| 8<br>Appropriate Applications                    | Any application where millisecond precision of requirements is acceptable, or at least non-critical | Any real-time application that is small enough to not warrant statecharts, REVS or RLP; for example, small process-control applications | Those particular parts of any application which are decision-intensive | See natural language                                                                    | Applications with strong data intensity or a need to identify cooperation between people; for example, business applications; many process-control applications | Any complex real-time application; especially those with sufficient complexity to warrant the multiple expressive power | Real-time applications that are stimulus-oriented; for example, defense systems                     | Real-time applications that are stimulus-oriented; for example, telephony                     | SDL was specifically developed for telephony systems, but is applicable to all RLP applications  | PAISLEY was developed for complex real-time systems                                                                             | Those particular parts of any application in which synchrony is important; specification of external behavior    |

FIGURE 28. A Comparison of SRS Approaches

### External View, Not Internal View

PDL does not provide help for the requirements writer who needs to remain at the requirements level and not proceed into design. Due to its genealogy (i.e., structured analysis, which is one of the worst offenders in this regard), SA/RT, must also score low. Finite state machines, decision tables, and decision trees require a considerable amount of human discipline to first define the external entities and then use the approach to describe the attributes of and existent relationships between those entities. Statecharts are slightly better as they provide the structural viewpoint independent from the behavioral and data-flow viewpoints. The methodology, however, does not encourage one to specifically define the requirements for an entity before defining that entity's internal structure. Finally, REVS, RLP, and SDL provide much better assistance due to their requirement to deal only with previously defined explicit external signals, although all three can be easily misused. REVS probably scores slightly lower than RLP and SDL because the view of "what will the system do in response to this input signal" is somewhat more of an inside-out view than the "what do I have to do to make the system perform this function" view exhibited by RLP and SDL.

### SRS Organizational Assistance

Finite state machines, decision tables, decision trees, PDL, and Petri-nets all provide rudimentary SRS organizational assistance. In particular, the FSM and Petri-net approaches offer the underlying machine model as the only organizational entity. Decision tables and trees provide the table and tree, respectively. PDL provides a module that can be used for anything. REVS enables SRS writers to organize the behavioral requirements into R-nets, each corresponding to a system stimulus. SDL and RLP enable SRS writers to organize the behavioral requirements into user-oriented features. SA/RT, statecharts, and PAISLey all provide at least two levels of organization because of their ability to separately model synchrony, data flow, and external behavior.

### Automatic Prototype and Test Generation

Automatic generation of prototypes is possible with FSMs (e.g., when using TDE [55-57]), statecharts (e.g., when using STATEMATE1 [34, 50]), and PAISLey, without much intervention other than defining the requirements in their respective languages. REVS provides prototype generation capability, but it requires considerable auxiliary coding effort on the user's part [31]. The feasibility of automatic prototype generation using RLP has been explored but not implemented [22, 53]. None of the other approaches support prototype generation. RLP has the capability to automatically generate system-level tests directly from the requirements [15].

### Appropriate Applications

Each technique is suitable to a particular subset of applications. The bottom row of Figure 28 describes the appropriate applications for each technique.

**Acknowledgments.** Dr. Edward Bersoff deserves special recognition for his faith, financial support, and moral support. Also, my thanks to Kathy Brown, Judy Israel, Marcia Lewis, Jere Milburn, and Lee Price for helping to make this a professional product.

### REFERENCES

1. Agerwala, T. Putting Petri-nets to work. *IEEE Comput.* 12, 12 (Dec. 1979), 85-94.
2. Alford, M. A requirements engineering methodology for real-time processing requirements. *IEEE Trans. Softw. Eng.* 3, 1 (Jan. 1977), 60-69.
3. Alford, M. SREM at the age of eight; The distributed design system. *IEEE Comput.* 18, 4 (Apr. 1985), 36-46.
4. Alford, M., and Burns, I. R-nets: A graph model for real-time software requirements. In *Proceedings of the Symposium on Computer Software Engineering*. Polytechnic Press, New York, 1976, pp. 97-108.
5. Basili, V., and Weiss, D. Evaluation of a software requirements document by analysis of change data. In *Proceedings of the 5th International Conference on Software Engineering* (San Diego, CA, March 9-12), pp. 314-323, IEEE Press, Wash., D.C., 1981.
6. Bell, T., and Bixler, D. A flow oriented requirements statement language. In *Proceedings of the Symposium on Computer Software Engineering*. Polytechnic Press, New York, 1976, pp. 109-122.
7. Berliner, E., and Zave, P. An experiment in technology transfer: PAISLey specification of requirements for an undersea lightwave cable system. In *Proceedings of the 9th IEEE International Conference on Software Engineering* (Monterey, CA, March 30-April 2), pp. 42-50, IEEE Press, Wash., D.C., 1987.
8. Berzins, V., and Gray, M. Analysis and design in MSG 84: Formalizing functional specifications. *IEEE Trans. Softw. Eng.* 11, 8 (Aug. 1985), 657-670.
9. Boehm, B. Software engineering. *IEEE Trans. Comput.* 25, 12 (Dec. 1976), 1226-1241.
10. Boehm, B., et al. Some experience with automated aids to the design of large-scale reliable software. *ACM SIGPLAN Not.* 10, 6 (June 1975), 105-113.
11. Buhrke, R., et al. Design choices for a large PCM switch with multi-processor control. In *Proceedings of the International Switching Symposium* (May 1979).
12. Caine, S., and Gordon, E. PDL—A tool for software design. In *Proceedings of the AFIPS National Computer Conference*, Vol. 44 (Anaheim, CA, 1975). AFIPS Press, Montvale, New Jersey, pp. 271-276.
13. Casey, B., and Taylor, B. Writing requirements in English: A natural alternative. In *Proceedings of the IEEE Software Engineering Standards Workshop* (San Francisco, California, August), IEEE Press, Wash., D.C., 1981, pp. 95-101.
14. Celko, J., et al. A demonstration of three requirements language systems. *SIGPLAN Not.* 18, 1 (Jan. 1983), 9-14.
15. Chandrasekharan, M., et al. Requirements-based testing of real-time systems: Modeling for testability. *IEEE Comput.* 18, 4 (Apr. 1985), 71-80.
16. Charette, R. *Software Engineering Environments*. McGraw Hill, New York, 1986.
17. Chvalovsky, V. Decision tables. *Softw. Pract. Exper.* 13 (1983), 423-429.
18. Daly, E. Management of software development. *IEEE Trans. Softw. Eng.* 3, 3 (May 1977), 230-242.
19. Dasarthy, B. Test plan generation for the requirements validation of real-time systems. In *Proceedings of the IEEE Workshop on Automatic Test Program Generation* (Philadelphia, PA, April), IEEE Press, Wash., D.C., 1986.
20. Davis, A. Requirements language processing for the effective testing of real-time software. *ACM Softw. Eng. Notes* 3, 5 (Nov. 1978), 61-66.
21. Davis, A. Automating the requirements phase: Benefits to later phases of the software life-cycle. In *Proceedings of IEEE COMPSAC '80* (Chicago, Ill., October 27-31), pp. 42-48, IEEE Press, Wash., D.C., 1980.
22. Davis, A. Rapid prototyping using executable requirements specifications. *ACM Softw. Eng. Notes* 7, 5 (Dec. 1982), 39-44.
23. Davis, A. The design of a family of applications-oriented requirements languages. *IEEE Comput.* 15, 5 (May 1982), 21-28.
24. Davis, A. The role of requirements in the software synthesis of real-time systems. In *Proceedings of the International Symposium on Current Issues in Requirements Engineering Environments* (Kyoto, Japan, Sept. 20-21) North-Holland, Amsterdam, The Netherlands, 1982, pp. 151-158.
25. Davis, A. A taxonomy for the early stages of the software development life cycle. *J. Syst. Softw.* 8, 4 (Sept. 1988).

26. Davis, A., et al. RLP: An automated tool for the processing of requirements. In *Proceedings of IEEE COMPSAC '79* (Chicago, Ill., November 5-8), pp. 289-299, IEEE Press, Wash., D.C., 1979.
27. Davis, A., and Rauscher, T. Formal techniques and automatic processing to ensure correctness in requirements specifications. In *Proceedings of the IEEE Specifications of Reliable Software Conference* (Cambridge, Mass., April), pp. 15-35, IEEE Press, Wash., D.C., 1979.
28. Davis, C., and Vick, C. The software development system. *IEEE Trans. Softw. Eng.* 3, 1 (Jan. 1977), 69-84.
29. DeMarco, T. *Structured Analysis and System Specification*. Prentice Hall, Englewood Cliffs, New Jersey, 1979.
30. Fagan, M. Design and code inspections and process control in the development of programs. IBM Report IBM-SDD-TR-21.572. Dec. 1974, IBM, Bethesda, Maryland.
31. Furia, N. A comparative evaluation of RSL/REVS and PSL/PSA applied to a digital flight control system. In *Proceedings of the AIAA 2nd Computers in Aerospace Conference*, pp. 330-337, American Institute of Aeronautics and Astronautics, New York, NY, 1979.
32. Glaseman, S., and Davis, M. Software requirements for embedded computers: A preliminary report. U.S. Air Force Document R-2567-AF. Mar. 1980.
33. Harel, D. Statecharts: A visual formalism for complex systems. *Sci. Comput. Prog.* 8 (1987).
34. Harel, D., et al. STATEMATE1: a working environment for the development of complex reactive systems. In *Proceedings of the Tenth International Conference on Software Engineering* (Singapore, April). IEEE Press, Wash. D.C., 1988.
35. Hatley, D. The use of structured methods in the development of large software-based avionics systems. In *Proceedings of the AIAA/IEEE 6th Digital Avionics Systems Conference* (Baltimore, MD), pp. 6-15, IEEE Press, Wash., D.C., 1984.
36. Heninger, K. Specifying software requirements for complex systems: New techniques and their application. *IEEE Trans. Softw. Eng.* 6, 1 (Jan. 1980), 2-13.
37. IEEE A guide for software requirements specifications. IEEE/ANSI Standard 830-1984. Institute of Electrical and Electronics Engineers, New York, 1984.
38. Kawashima, H., et al. Functional specification of call processing by state transition diagram. *IEEE Trans. Commun.* 19, 5 (Oct. 1971), 581-587.
39. Kerola, P., and Freeman, P. A comparison of life cycle models. In *Proceedings of the 5th International Conference on Software Engineering* (San Diego, CA, March 9-12), pp. 90-99, IEEE Press, Wash., D.C., 1981.
40. Miller, T., and Taylor, B. A system requirements methodology. In *Proceedings of the IEEE Electro '81 Conference* (New York, Apr.), pp. 18.5.1-18.5.5, IEEE Press, Wash., D.C. 1981.
41. Moret, B. Decision trees and diagrams. *ACM Comput. Surv.* 14, 4 (Dec. 1982), 593-623.
42. Patil, S. Coordination of asynchronous events. Ph.D. dissertation. MIT, Cambridge, Massachusetts, 1970.
43. Peterson, J. Petri-nets. *ACM Comput. Surv.* 9, 3 (Sept. 1977), 223-252.
44. Petri, C. Kommunikation mit automation. *Schriften des Reinsch-Westfälischen Inst. für Instrumentelle Mathematik an der Universität Bonn*, Bonn, West Germany, 1962.
45. Pressman, R. *Software Engineering: A Practitioner's Guide*. McGraw Hill, New York, 1982.
46. Rockstrom, A., and Saracco, R. SDL—CCITT specification and description language. *IEEE Trans. Commun.* 30, 6 (June 1982), 1310-1318.
47. Roman, G.-C., et al. A total system design framework. *IEEE Comput.* 17, 5 (May 1984), 15-26.
48. Royce, W. Managing the development of large software systems: Concepts and techniques. Reprinted in *Proceedings of the Ninth International Conference on Software Engineering* (Monterey, Calif., March 30-April 2), pp. 328-338, IEEE Press, Wash., D.C., 1987.
49. Searle, L. An air force guide to the system specification. U.S. Air Force Document TM-5772/010/00. Jan. 1981.
50. STATEMATE1: THE STATEMATE1 Working Environment for System Development. AD CAD, Inc., Cambridge, Massachusetts, 1985.
51. Sunshine, C. Formal methods for communication protocol specification and verification. ARPA Order 3460/3681. Nov. 1979.
52. Taylor, B. A method for expressing functional requirements of real-time systems. In *Proceedings of the IFAC/IFIP Workshop on Real-Time Programming* (Leibnitz, Austria, April), Pergamon Press, New York, 1980.
53. Wang, Y. A distributed specification model and its prototyping. In *Proceedings of IEEE COMPSAC '86* (Chicago, Ill., October 8-10), pp. 130-137, IEEE Press, Wash., D.C., 1986.
54. Ward, P. The transformation schema: An extension of the data flow diagram to represent control and timing. *IEEE Trans. Softw. Eng.* 12, 2 (Feb. 1986), 198-210.
55. Wasserman, A. User software engineering and the design of interactive systems. In *Proceedings of the Fifth IEEE International Conference on Software Engineering* (San Diego, March 9-12), pp. 387-393, IEEE Press, Wash., D.C., 1981.
56. Wasserman, A. Extending state transition diagrams for the specification of human-computer interaction. *IEEE Trans. Softw. Eng.* 11, 8 (Aug. 1985), 699-713.
57. Wasserman, A., et al. Developing interactive information systems with the user software engineering methodology. *IEEE Trans. Softw. Eng.* 12, 2 (Feb. 1986), 326-345.
58. Whitis, V., and Chiang, W. A state machine development method for call processing software. In *Proceedings of the IEEE Electro 81* (Apr.) IEEE Press, Wash., D.C., 1981.
59. Yoeli, M., and Barzilai, Z. Behavioural descriptions of communication switching systems using extended Petri-nets. *Digital Processes* 3, 4 (1977), 307-320.
60. Zave, P. An operational approach to requirements specification for embedded systems. *IEEE Trans. Softw. Eng.* 8, 3 (May 1982), 250-269.
61. Zave, P., and Shell, W. Salient features of an executable specification language and its environment. *IEEE Trans. Softw. Eng.* 12, 2 (Feb. 1986), 312-325.
62. Zave, P., and Yeh, R. Executable requirements for embedded systems. In *Proceedings of the 5th IEEE International Conference on Software Engineering* (San Diego, CA, March 9-12), pp. 285-304, IEEE Press, Wash., D.C., 1981.

**CR Categories and Subject Descriptors:** D.2.1 [Software Engineering]: Requirements/Specifications; D.3.2 [Programming Languages]: Language Classifications

**General Terms:** Requirements Tools, Software Engineering Tools, Software Requirements Specifications

**Additional Key Words and Phrases:** Decision tables, decision trees, finite state machines, PAISLEY, PDL, Petri nets, REVS, RPL, SDL, statecharts, structured analysis

Received 11/87; accepted 3/88

#### ABOUT THE AUTHOR:

**ALAN M. DAVIS** is vice president and director of engineering services at BTG, Incorporated, in Vienna, Virginia. He is also chair of the IEEE Computer Society's Working Group on Software Requirements Standards. He has authored numerous publications in the areas of software engineering tools, requirements specification techniques and languages. His current address is: Alan M. Davis, BTG, Inc., 1945 Old Gallows Road, Vienna, VA 22180.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.